🔬 Food Nutrition Estimation from RGB Image: Complete Technical Guide

A comprehensive explanation of the methodology, metrics, features, equations, and results for your conference presentation.

📋 Table of Contents

1. Problem Statement & Motivation
2. Evaluation Metrics (MAE, RMSE, MAPE)
3. The 9 Geometric Features (Detailed Breakdown)
4. Mathematical Equations & Paths
5. Model Architecture
6. Development Phases 3-6
7. Uncertainty Quantification (MC Dropout)
8. Results & Interpretation
9. Error Patterns & Limitations
10. Deployment & User Interface

1. Problem Statement & Motivation

The Core Challenge

Nutritionists and fitness enthusiasts need accurate calorie and macronutrient estimates for food. Traditionally, this requires:

Food weighing: Precise kitchen scales (cumbersome for daily meals)

Manual lookup: USDA database searches (time-consuming)

Eyeballing portions: Error-prone (±50% uncertainty typical)

Your Innovation: Estimate nutrition from a **single RGB photograph** using computer vision + machine learning, with **quantified uncertainty**.

Why Visual Appearance Alone Isn't Enough

Example: A bowl of pasta looks identical whether it weighs 200g (dry weight) or 500g (cooked weight). The cooked version has ~150% more calories. Visual features alone cannot distinguish between them.

Your Solution: Combine three information sources:

RGB texture: Learn food type patterns (e.g., tomato sauce is red)

Segmentation mask: Determine the boundary of the food item

Depth map: Estimate 3D structure, volume, and mass

2. Evaluation Metrics: MAE, RMSE, MAPE

Why Multiple Metrics?

Different metrics capture different failure modes. You use three:

MAE (Mean Absolute Error)

$$\text{MAE} = \frac{1}{N}\sum_{i=1}^{N} y_i - \hat{y}_i$$

What it means: Average prediction error in the same units as the target (kcal, grams, etc.)

Example: MAE = 119.79 kcal means on average your prediction is off by ±120 calories.

Why use it: Intuitive, robust to outliers (doesn't penalize extreme errors as harshly).

When it matters: For user-facing apps, MAE feels fair—big errors count the same as small ones.

RMSE (Root Mean Squared Error)

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}\left(y_i - \hat{y}_i\right)^2}$$

What it means: Penalizes large errors more heavily than small ones.

Example: RMSE = 158.69 kcal vs MAE = 119.79 kcal. The gap (39 kcal) indicates you have some larger prediction errors.

Why use it: Encourages the model to avoid extreme failures.

When it matters: Safety-critical applications. RMSE > MAE signals high variance.

MAPE (Mean Absolute Percentage Error)

$$\text{MAPE} = \frac{100\%}{N} \sum_{i=1}^{N} \frac{y_i - \hat{y}_i}{y_i}$$

What it means: Relative error (percentage). Independent of scale.

Example: Predicting 100 kcal when actual is 200 kcal = 50% error.

Why you use it secondarily: Can be unstable when true values are small (division by near-zero).

Your Metric Hierarchy (Stated on Poster)

Rank	Metric	Reason
Primary	MAE & RMSE	Stable, actionable in real units (kcal, grams)
Secondary	MAPE	Provides relative perspective; less reliable for small values

Interpretation of Your Results

Phase 4 (Direct Nutrient)

Calories MAE: 119.79 kcal

For a 500 kcal meal, ±119 kcal = ±24% error.

Status: Good for casual tracking, not medical-grade.

Phase 6 (Weight-First)

Weight MAE: 98.9 g

For a 200g portion, ±99g = ±49% error.

Status: Weight prediction is noisy; nutrient conversion adds error.

3. The 9 Geometric Features (Detailed Breakdown)

Definition: A 9-dimensional vector $g = [f_1, f_2, ..., f_9]$ extracted from the food mask and depth map. These replace raw pixels as input to the nutrition MLP.

Why Geometry Instead of Pixels?

Scale invariance: A 100g meatball and a 500g meatball look similar in RGB; geometry captures the difference.

Physical grounding: Calories correlate with mass/volume, not appearance.

Dimensionality reduction: From 1.2M pixels → 9 numbers; easier to learn from limited data.

The 9 Features (Exact Definitions)

1. Mask Area (pixels)

Computation: Sum of all pixels in the segmentation mask.

Formula: `area = count(mask > 0.5)`

Interpretation: Larger food footprint in image → likely more food mass.

Range: 1,000 – 500,000 pixels (depends on camera distance).

2. Mask Perimeter (pixels)

Computation: Boundary length of the food contour.

Method: Trace the edge of the mask, sum edge lengths.

Interpretation: Complex, convoluted foods (e.g., pasta) have high perimeter.

Signal: Shape complexity can hint at density variations.

3. Convex Hull Area (pixels)

Computation: Area of the smallest convex shape enclosing the mask.

Method: Convex hull algorithm (Graham scan).

Interpretation: Ratio of (mask area / convex hull area) = shape solidity.

Example: Soup (solid) → ratio ≈ 1; Spaghetti (complex) → ratio ≈ 0.4.

4. Mean Depth (raw units)

Computation: Average depth value within the mask.

Formula: `mean_depth = mean(depth[mask > 0.5])`

Interpretation: Higher mean depth = thicker/taller food item.

Why it matters: Two items with same area but different heights have different volumes.

5. Std Dev Depth (raw units)

Computation: Standard deviation of depth within the mask.

Formula: `std_depth = std(depth[mask > 0.5])`

Interpretation: High std dev = uneven food (e.g., chunky stew, layered cake).

6. Min Depth (raw units)

Computation: Smallest depth value in the mask (closest point to camera).

Interpretation: Peaks/protrusions toward camera.

Example: A sandwich has low min_depth (bread peak); soup has high

Signal: Captures structural heterogeneity → can hint at ingredient mixing.

min_depth (flat surface).

7. Max Depth (raw units)

Computation: Largest depth value in the mask (farthest point from camera).

Interpretation: Overall food thickness/height.

Range: Larger range (max - min) = taller, more 3D food.

8. Depth Range (raw units)

Computation: `depth_range = max_depth - min_depth`

Interpretation: Total vertical extent of the food.

Signal: Range ≈ 0 = flat food (plate of cookies); Range >> 0 = tall food (glass of juice).

9. Solidity Index (unitless)

Computation: `solidity = mask_area / convex_hull_area`

Range: 0 to 1. (1 = perfect circle/square; 0.5 = highly concave).

Interpretation: Compact foods (fruit) → solidity near 1. Complex foods (pasta, salad) → low solidity.

Biological signal: Compact → dense; complex → airy or mixed.

How 9D Features Replace Raw Pixels

```
Input RGB image (1280×720×3 = 2.76M values)
↓
YOLOv8-seg predicts food mask (binary 1280×720 = 921.6K values)
↓
MiDaS predicts depth map (1280×720 = 921.6K values)
```

```
                                        ↓
                Compute 9 geometric statistics from mask & depth
                                        ↓
      9D feature vector g = [area, perimeter, conv_hull, mean_d, std_d, min_d,
                            max_d, range_d, solidity]
                                        ↓
                      Feed to MLP for nutrition prediction
```

Advantages of This Approach

Explainability: You can inspect each feature; understand why prediction high/low.

Robustness to lighting: Geometry doesn't care if food is in shadow or bright sunlight.

Data efficiency: 9 features from 2.76M pixel values; trains with fewer samples.

Transfer learning: YOLOv8 & MiDaS are pre-trained on millions of images.

4. Mathematical Equations & Prediction Paths

The Two Nutrient Estimation Paths

Your system computes nutrition via two independent methods. Both take the 9D geometry vector as input.

Path A: Direct Nutrient Regression (Phase 4)

$$\hat{\mathbf{n}} = f_\theta(\mathbf{g})$$

where:

g = 9D geometric feature vector

f = MLP neural network (128 → 64 → 32 layers)

θ = learned weights and biases

n̂ = predicted nutrient vector = [ĉ, f̂, p̂, ĉ$_{ar}$b] (4 macronutrients)

Detailed Explanation: Direct Path

Input: 9 numbers (geometry)

Layer 1 (Input → 128): `h₁ = ReLU(BatchNorm(W₁g + b₁))`
Maps 9D geometry to 128 hidden units. ReLU adds non-linearity (enables learning complex shapes).

Layer 2 (128 → 64): `h₂ = ReLU(BatchNorm(W₂h₁ + b₂)) with Dropout(0.2)`
Compresses to 64 units. Dropout randomly zeros 20% of activations during training → prevents overfitting.

Layer 3 (64 → 32): `h₃ = ReLU(BatchNorm(W₃h₂ + b₃)) with Dropout(0.2)`
Further compression to 32 units.

Output Layer (32 → 4): `n̂ = Linear(h₃) = [W₄h₃ + b₄]`
4 nutrient predictions (no activation, unbounded regression).

Training loss:

$$\mathcal{L} = \mathrm{MAE}(\hat{\mathbf{n}}, \mathbf{n}) = \frac{1}{4}\sum_{j=1}^{4} |\hat{n}_j - n_j|$$

Mean error across all 4 nutrients. Model learns to minimize prediction error on training data.

Path B: Weight-First Conversion (Phase 6)

$$\hat{w} = h_\phi(\mathbf{g})$$

$$\hat{n}_j = \hat{w} \cdot \rho_j \quad \text{for } j \in \{\text{cal, fat, prot, carb}\}$$

Detailed Explanation: Weight-First Path

Step 1: Predict Weight (using same 9D geometry)
`ŵ = h_φ(g)` where h_ϕ is another MLP (structure: 128 → 64 → 32 → 1)
Output: Single scalar = predicted weight in grams

Example: If actual food is pasta, model predicts ŵ = 180g.

Step 2: Convert Weight to Nutrients Using Constants
Once you have weight, multiply by per-gram constants (ρ_j):

$$\hat{c} = \hat{w} \times \rho_{\text{cal}} \quad \text{(calories per gram)}$$

$$\hat{f} = \hat{w} \times \rho_{\text{fat}} \quad \text{(fat per gram)}$$

$$\hat{p} = \hat{w} \times \rho_{\text{prot}} \quad \text{(protein per gram)}$$

$$\hat{c}_{arb} = \hat{w} \times \rho_{\text{carb}} \quad \text{(carbs per gram)}$$

Example (Pasta):

> Predicted weight: 180g

Pasta nutritional constant: 1.3 cal/g (dry basis)

Predicted calories: 180g × 1.3 = 234 kcal

Where do ρ values come from?

First, classify food type (Phase 5: EfficientNet → Top-5 dishes)

Query USDA database for average macros of that dish type

If available in cache: use precomputed constants

If not cached: query USDA API (slow, ~1s per food type)

Why Two Paths?

Path A: Direct (Phase 4)

Pros: Fast end-to-end, learns implicit relationships.

Cons: "Black box"—hard to debug why prediction high/low.

Path B: Weight-First (Phase 6)

Pros: Interpretable (humans understand weight → calories). Uses external priors.

Cons: Two stages of error; errors accumulate.

Your strategy: Compute both, report both, let user choose (or ensemble them).

5. Model Architecture (Detailed)

System Overview

```
Input image (1280×720)
↓
[YOLOv8n-seg] → Food mask (binary)
↓
[MiDaS Small] → Depth map
↓
[Geometry Extractor] → 9D feature vector g
↓
[Phase 4 MLP] → Nutrition (4 outputs)
[Phase 5 EfficientNet-B0] → Food class (Food-101 top-5)
[Phase 6 MLP] → Weight (1 output)
↓
[USDA Lookup] (if using Phase 6)
↓
[Gradio UI] → Display with confidence intervals
```

Component 1: YOLOv8n-seg (Segmentation)

Property	Value	Meaning
Model	YOLOv8n-seg	"n" = nano (smallest, fastest variant)
Task	Instance segmentation	Predicts pixel-level mask of food vs. background
Input size	1280×720 (or 640×640)	You provide RGB image

Output	Binary mask (0 = background, 1 = food)	Or soft mask (0–1 probabilities)
Speed	~30 FPS on GPU	Real-time capable
Pre-trained on	COCO 80 classes	Generic object segmentation; fine-tuned on your food data

Why YOLOv8n? Balance speed vs. accuracy. Larger variants (medium, large) are more accurate but slow.

Component 2: MiDaS (Depth Estimation)

Property	Value	Meaning
Model	MiDaS Small	Monocular Depth Estimation; "Small" = lightweight
Input	RGB image	Same RGB as segmentation input
Output	Relative depth map (1280×720)	Pixel-wise depth (not absolute cm; relative ordering only)
Pre-trained on	NYU Depth v2, KITTI, etc.	Indoor and outdoor scenes; generalizes to food
Speed	~50 FPS on GPU	Faster than segmentation

Why depth? RGB alone doesn't tell you volume. Two identical-looking bowls of soup can have very different depths (and thus volumes and calories).

Component 3: Geometry Extractor (Your Custom Code)

Takes mask + depth → outputs 9D feature vector. This is **differentiable** (all operations

Takes mask + depth → outputs 9D feature vector. This is **differentiable** (all operations are, so gradients can flow back).

Component 4: Phase 4 MLP (Direct Nutrition)

Layer	Input Dim	Output Dim	Activation	Regularization
1 (Input)	9	128	ReLU	BatchNorm
2	128	64	ReLU	BatchNorm + Dropout(0.2)
3	64	32	ReLU	BatchNorm + Dropout(0.2)
4 (Output)	32	4	None (Linear)	None

Total parameters: ~15K (tiny model, fits any device).

Component 5: Phase 6 MLP (Weight Prediction)

Same architecture as Phase 4, but output layer outputs 1 value (weight) instead of 4 (nutrients).

Component 6: Phase 5 EfficientNet-B0 (Food Classification)

Property	Value	Meaning
Model	EfficientNet-B0	Lightweight CNN; "B0" = smallest scale
Dataset	Food-101 (101 food categories)	Identifies what type of food (pizza, salad, sushi, etc.)

Output	Top-5 class probabilities	E.g., [sushi: 85%, salmon: 10%, rice: 3%, ...]
Parameters	~5.3M	Larger than MLPs but still efficient
Used for	USDA lookup prior	If Phase 6 weight prediction, use food class to look up nutritional constants

Component 7: USDA Cache/API

Stores precomputed nutritional constants for common foods. Examples:

Food	ρ_cal (kcal/g)	ρ_fat (g/g)	ρ_prot (g/g)	ρ_carb (g/g)
Pasta (dry)	3.7	0.013	0.125	0.75
Chicken breast (cooked)	1.65	0.036	0.314	0
Olive oil	8.84	1.0	0	0

Your system caches these so Phase 6 doesn't need to query USDA API every time (slow).

6. Development Phases 3–6 (Iterative Refinement)

Why Phases?

You built incrementally. Each phase added complexity, tested new ideas, measured improvement.

Phase 3: ResNet-50 Baseline

Aspect	Details
Input	Raw RGB image (1280×720)
Model	ResNet-50 pretrained on ImageNet, fine-tuned output layer for 4 nutrients
Output	Direct nutrient predictions from pixels
MAE (calories)	**194.25 kcal** ← Baseline error
Insight	Pixels alone don't encode volume/mass well. Similar-looking foods vary widely.

Phase 4: Geometry-Based MLP (Direct Nutrition)

Aspect	Details
Input	9D geometric feature vector (from YOLOv8 + MiDaS)
Model	MLP (128 → 64 → 32 → 4)
Output	Direct nutrient predictions
MAE (calories)	**119.79 kcal** ← 38.3% improvement! 🎯
RMSE	

RMSE (calories)	158.69 kcal
Key insight	Geometry (area, depth, shape) is far more predictive than raw pixels for nutrition.
Per-nutrient MAE	Fat: 7.67g, Protein: 11.01g, Carb: 11.14g

Phase 5: Food Classification (EfficientNet-B0)

Aspect	Details
Input	RGB image
Model	EfficientNet-B0 trained on Food-101
Output	Top-5 predicted food classes with probabilities
Accuracy (test)	Top-1: 85.86%, Top-5: 97.03%
Purpose in pipeline	Provides **prior** for USDA lookup (Phase 6). Helps user understand what model thinks the food is.
Example output	"95% sushi, 4% salmon nigiri, 1% rice bowl"

Phase 6: Weight-First MLP (Indirect Nutrition)

Aspect	Details
Input	9D geometric feature vector (same as Phase 4)

Step 1	MLP predicts weight in grams (128 → 64 → 32 → 1)
Step 2	Look up food class (Phase 5), query USDA constants ρ_j
Step 3	Convert: nutrient = weight × ρ_j
Weight MAE (test)	98.9 grams
Derived nutrient MAE (calories)	133.8 kcal (slightly worse than Phase 4 direct)
Why test it?	Weight prediction is more **interpretable**

Performance Summary Across Phases

Phase	Approach	Cal. MAE (kcal)	Cal. RMSE (kcal)	Improvement vs Phase 3
3	ResNet-50 (pixels)	194.25	—	Baseline
4	Geometry MLP (direct)	**119.79**	158.69	**-38.3%** 🎯
5	EfficientNet (classification only)	—	—	Prior for Phase 6
6	Weight MLP + USDA conversion	133.8	—	-31.1% (competitive)

Conclusion: Phase 4 geometry-based direct prediction is **best**. Phase 6 is competitive and more interpretable, so you report both.

7. Uncertainty Quantification (MC Dropout)

Why Report Uncertainty?

Don't just give a point estimate ("200 kcal"). Users need to know confidence ("200 ± 50 kcal" = 68% confident in range).

MC Dropout Explained

Core idea: During inference, keep dropout enabled (normally it's only during training). Run the model T times with different random dropout patterns, then aggregate results.

For each forward pass $t \in \{1, ..., T\}$:

$$\hat{y}^{(t)} = f_\theta(\mathbf{g}, \text{dropout enabled})$$

Compute empirical mean and variance:

$$\mu = \frac{1}{T} \sum_{t=1}^{T} \hat{y}^{(t)}$$

$$\sigma^2 = \frac{1}{T} \sum_{t=1}^{T} (\hat{y}^{(t)} - \mu)^2$$

Your Settings

App inference: T = 30 samples (balance speed vs. accuracy)

Notebook evaluation: T = 50 samples (more thorough analysis)
Reported intervals: μ ± σ (68% confidence) and μ ± 2σ (95% confidence)

Example Output

User uploads photo of pasta. Your system outputs:

```
Predicted Calories: 250 kcal
68% Confidence: 220–280 kcal (μ ± σ)
95% Confidence: 190–310 kcal (μ ± 2σ)
Model uncertainty (std): 30 kcal
```

Interpretation for user: "Most likely 250 kcal, but could be anywhere from 190–310 kcal. That pasta is noisy to predict."

Why MC Dropout Works

Different dropout masks → network takes slightly different paths → outputs vary → variation ≈ model uncertainty.

If all T samples give similar output, uncertainty is low (model confident). If samples vary widely, uncertainty is high (model unsure).

Low-Confidence Suppression

Safety feature: If σ is very large (high uncertainty), you can optionally suppress the prediction and flag it for manual review instead.

Example: "Unable to confidently predict (high uncertainty). Please verify or edit manually."

8. Results & Interpretation

Test Set Performance (Phase 4 Direct)

Nutrient	MAE	RMSE	Typical meal size	% Error
Calories	119.79 kcal	158.69 kcal	~500 kcal meal	±24%
Fat	7.67 g	10.66 g	~30g in meal	±26%
Protein	11.01 g	15.75 g	~35g in meal	±31%
Carbs	11.14 g	14.35 g	~60g in meal	±19%

Interpretation

Best performance: Carbs (±19%), then Calories (±24%). Likely because both correlate with volume/density.

Worst performance: Protein (±31%). Why? Protein varies wildly by food (tofu vs. chicken vs. legumes look similar but have different protein). Geometry can't distinguish.

Use case: Good for casual tracking (±20% error is acceptable). Not medical-grade nutrition therapy.

Why Phase 4 > Phase 6?

Phase 4 direct: 119.79 kcal MAE

Phase 6 weight-first: 133.8 kcal MAE (12% worse)

Reason: Phase 6 has two error sources:

1. **Weight prediction error:** ±99g (as reported)
2. **USDA constant error:** Constants are averages; real food may have different density

Phase 4 skips the weight step, allowing the MLP to learn implicit nonlinear relationships (e.g., "this specific geometry pattern means higher carb density than average").

Phase 5 Classification Performance

EfficientNet-B0 on Food-101:

Top-1 accuracy: 85.86% (correct prediction 85% of time)

Top-5 accuracy: 97.03% (correct answer in top 5 predictions 97% of time)

Best epoch: 23 (then starts overfitting)

What this means: Food classification is pretty reliable. Users trust the "what is this?" answer 97% of the time. Helps with USDA lookup (Phase 6) and debugging.

9. Error Patterns & Limitations

When Does the System Fail?

Error Pattern 1: Mixed Dishes with Overlapping Components

Example: Salad with various vegetables, dressing, cheese.

Problem: YOLOv8-seg treats entire salad as one mask. Geometry features average over all components. Model doesn't know how much of the mask is lettuce (low cal) vs. croutons (high cal).

Failure mode: Predicts average nutrition; could be off by ±50% for individual components.

Error Pattern 2: Strong Occlusion or Unusual Viewpoints

Example: Food partially hidden behind a fork, or shot from extreme angle.

Problem: YOLOv8 mask becomes incomplete. Depth map breaks down (can't see through occluder). Geometry features extrapolate wrongly.

Failure mode: Underestimates food mass; predicts too few calories.

Error Pattern 3: Similar Geometry, Different Density

Example: A bowl of popcorn vs. a bowl of rice (both white, similar volume).

Problem: Geometry alone can't distinguish air pockets from dense grain. Popcorn is ~90% air (low cal/g), rice is ~10% air (high cal/g).

Failure mode: Predicts same calories for both; major error for popcorn (50% underestimate) and rice (50% overestimate).

Error Pattern 4: Extreme Aspect Ratios

Example: Thin crepe vs. thick cake (same footprint, different height).

Problem: Depth range (max - min) captures thickness, but model may struggle to learn the nonlinear relationship (thick ≠ always means more calories if it's airy).

Failure mode: Occasional mispredictions for extreme aspect ratios.

Mitigation Strategies

Uncertainty reporting: Flag high-uncertainty predictions for manual review.

User edits: Gradio UI allows users to correct predictions ("I said it's 250g but model predicted 180g").

Food classification: Phase 5 helps user verify ("Is this what you meant by pizza?").

Ensemble methods: Report both Phase 4 and Phase 6; let user choose or average.

10. Deployment & User Interface

Gradio UI Workflow

```
User uploads photo
↓
[YOLOv8 + MiDaS] Segment & depth extraction (~1-2 sec)
↓
[Phase 5] Classify food (~0.5 sec)
↓
[Phase 4] Direct nutrient prediction with MC Dropout (~1 sec)
[Phase 6] Weight prediction + USDA lookup (~1 sec)
↓
[Display Results]
- Result table (nutrients + confidence intervals)
- Classification top-5
- Food image with segmentation mask overlay
↓
User can edit nutrients & resubmit
```

Output Example

Input: Photo of grilled chicken with roasted vegetables.

Output displayed to user:

Nutrient	Predicted	68% CI	95% CI
Calories	380 kcal	350–410	320–440
Fat	12g	10–14g	8–16g
Protein	52g	46–58g	40–64g
Carbs	15g	13–17g	11–19g

Food classification: 92% chicken, 6% vegetables, 2% other

Segmentation visualization: [Image with green mask overlay showing detected chicken & veggies]

User Editing Panel

If user disagrees: "Actually, I added extra sauce. It's ~450 kcal, not 380."

Gradio allows them to edit each nutrient → stored in database → can fine-tune model later.

Benefits of This Workflow

Human-in-the-loop: Model + human judgment = better outcomes.

Transparency: User sees confidence intervals → understands model uncertainty.

Data collection: User edits become training data for future model improvements.

Safety: Low-confidence predictions flagged (don't let model make risky guesses).

Performance Targets

Speed: 9sec end-to-end (YOLOv8 + depth + MLP inference)

Accuracy: ±120 kcal MAE (24% relative error on 500 kcal meal)

Availability: Can run on GPU or CPU (slow on CPU but possible)

Summary: Key Takeaways for Your Presentation

🎯 Core Innovation

Replace raw pixels with **9 geometric features** (area, perimeter, depth statistics, solidity). This 39% error reduction proves geometry matters more than appearance.

📊 Key Results

Phase 4 MAE: 119.79 kcal (vs. 194.25 ResNet baseline)

Phase 5 accuracy: 85.86% top-1 food classification

Phase 6 weight MAE: 98.9 grams

🔧 Technical Highlights

- YOLOv8n segmentation + MiDaS depth → 9D features → MLP nutrition
- MC Dropout for uncertainty quantification (T=30 app, T=50 notebook)
- Two prediction paths: direct (Phase 4) and weight-first (Phase 6)
- Phase 5 food classification (EfficientNet-B0, 97% top-5)

⚠️ Limitations

- Mixed dishes (hard to segment individual components)
- Occlusion (can't see behind objects)
- Similar geometry, different density (e.g., popcorn vs. rice)
- ±24% relative error (not medical-grade)

🚀 Real-World Impact

Practical meal tracking app: take photo → instant nutrition + editable output. Useful for fitness, diet tracking, meal planning. Uncertainty intervals help users make informed decisions.